

DEVELOPER REFERENCE MANUAL

HealthBridge API Documentation

AI-Powered Community Health Information & Support
Platform

LoopLearn Hackathon 2026

Version v1.0

Developer documentation for the HealthBridge API ecosystem, including authentication, health resources, AI services, resource intelligence, and supporting microservices.

Table of Contents

01 Introduction	3
02 System Overview	4
03 API Architecture	5
04 Base Architecture & Services	7
05 Authentication & Authorization	8
06 API Gateway	10
07 Authentication Service	11
08 Resource Service	12
09 AI Service	13
10 Health Resource Agent	14
11 Python FastAPI Service	15
12 RAG System	16
13 Request & Response Standards	17
14 Error Handling	18
15 Security	19
16 Example API Workflows	21
17 Environment Configuration	24
18 Running the API	25
19 API Service Ports	26
20 Future API Improvements	27
21 Conclusion	28

SECTION 01

Introduction

HealthBridge is an AI-powered platform designed to connect citizens with verified health resources and educational content.

This documentation describes the HealthBridge API ecosystem: the backend services, the way the frontend communicates with those services, and the mechanisms that keep the platform secure, extensible, and easy to reason about. It is written for engineers who need to integrate with, extend, or operate the HealthBridge backend.

Across the following sections, this manual:

- Explains the backend architecture and how each microservice fits into the overall system
- Describes communication patterns between the frontend and backend services
- Documents authentication and authorization, including JWT- and cookie-based session handling
- Explains how health resources and educational content are created, managed, and approved
- Explains the AI functionality that powers the Health Education Assistant
- Describes the Retrieval-Augmented Generation (RAG) system used to ground AI responses
- Documents the Health Resource Agent and its tool-based execution model
- Explains the role of the Python FastAPI service in resource analysis and search

NOTE This document is derived from the HealthBridge project overview supplied for the LoopLearn Hackathon 2026. Where exact endpoint-level detail (methods, parameters, payload schemas) was not present in the source material, the corresponding section says so explicitly rather than inventing specifications.

SECTION 02

System Overview

HealthBridge serves three distinct user roles, each with a different set of capabilities exposed through the same underlying API surface.

Citizen

- Search health resources
- Browse health education content
- Ask health questions
- Use the AI Health Education Assistant

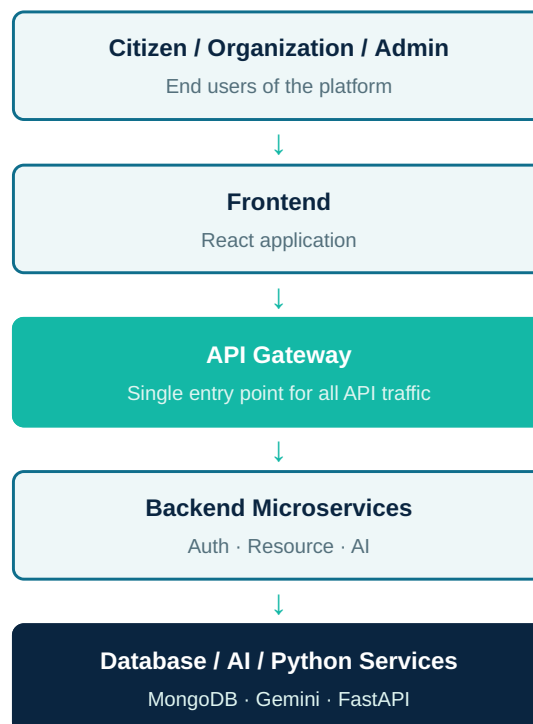
Organization

- Create health resources
- Manage health resources
- Add educational content
- Track resource status

Admin

- Manage users
- Manage content
- Approve or reject resources
- View platform statistics

Platform Flow

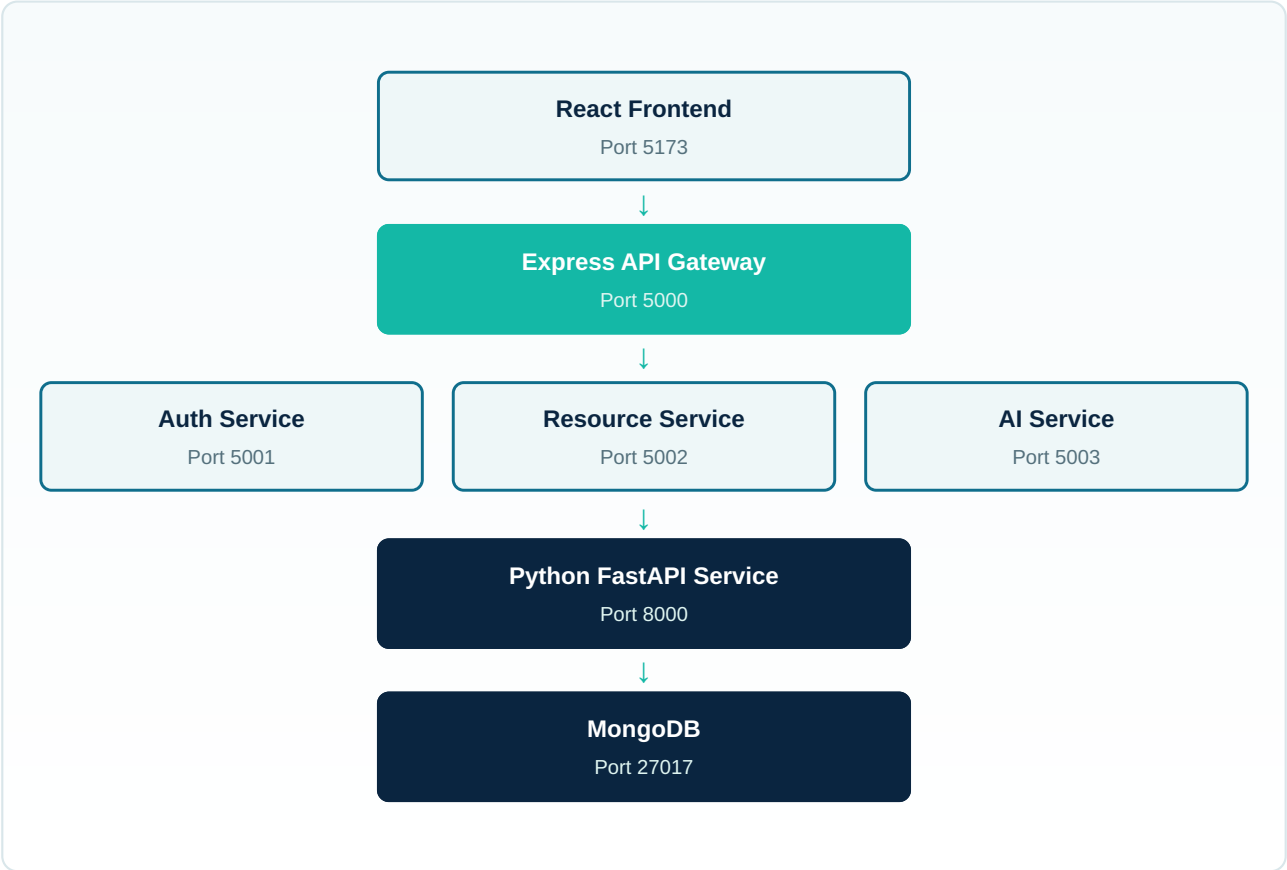


Every request, regardless of which role initiates it, follows the same path: it originates in the React frontend, is routed through the API Gateway, and is dispatched to the appropriate backend microservice, which in turn may call the database, the AI provider, or the Python analysis service.

SECTION 03

API Architecture

HealthBridge is built as a set of small, independently-run services rather than a single monolithic backend. The diagram below reflects the documented architecture.



Responsibility of Each Layer

Layer	Responsibility
React Frontend	Renders the citizen, organization, and admin experiences and issues requests to the API Gateway.
API Gateway	Single entry point for all frontend traffic; forwards requests to the correct downstream microservice.
Auth Service	Handles authentication, password security, and issuance of JWTs.
Resource Service	Owns health resources and articles, including creation, status, and approval workflows.
AI Service	Powers the Health Education Assistant, the RAG pipeline, and the Health Resource Agent.
Python FastAPI Service	Performs resource analysis, classification, and relevance scoring.

MongoDB

Persists users, resources, articles, and related platform data.

NOTE Only services documented in the HealthBridge project overview are represented here; no additional services have been added.

SECTION 04

Base Architecture & Services

The table below is the canonical service directory for HealthBridge: the technology each service is built with, what it is responsible for, and the port it listens on in the reference Docker Compose setup.

Service	Technology	Responsibility	Port
Frontend	React / Vite	User interface	5173
API Gateway	Express.js	API entry point	5000
Auth Service	Node.js / Express	Authentication	5001
Resource Service	Node.js / Express	Resources & articles	5002
AI Service	Node.js / Express	AI, RAG & Agent	5003
Python Service	FastAPI	Resource analysis	8000
Database	MongoDB	Persistent data	27017

Frontend

:5173

React / Vite user interface

API Gateway

:5000

Express.js entry point

Auth Service

:5001

Node.js / Express authentication

Resource Service

:5002

Node.js / Express resources

AI Service

:5003

Node.js / Express AI, RAG & agent

Python Service

:8000

FastAPI resource analysis

MongoDB

:27017

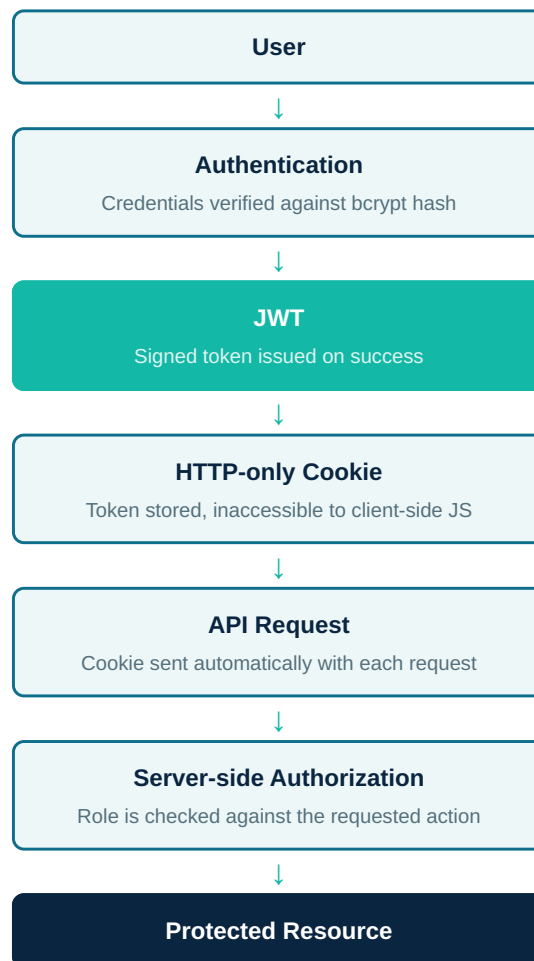
Persistent data store

SECTION 05

Authentication & Authorization

HealthBridge secures access to the platform using a combination of JWT-based authentication, bcrypt password hashing, HTTP-only cookies, and server-side role-based authorization.

Authentication Flow



Roles

Three roles are recognized platform-wide: **Citizen**, **Organization**, and **Admin**. Role checks are enforced server-side; the role encoded in the session determines which actions and resources a request may access.

Authentication Endpoints

Endpoint details should be added from the complete API specification. The project overview confirms JWT issuance, bcrypt-based credential verification, and HTTP-only cookie storage, but does not enumerate exact route paths, request payloads, or response schemas for authentication endpoints.

Once available, each authentication endpoint should be documented using the following format:

Endpoint Name

METHOD Endpoint: `/example`

Authentication: Required / Not Required

Role: Citizen / Organization / Admin

Description: Explain the endpoint.

Request

```
{
  "example": "value"
}
```

Response

```
{
  "success": true
}
```

SECTION 06

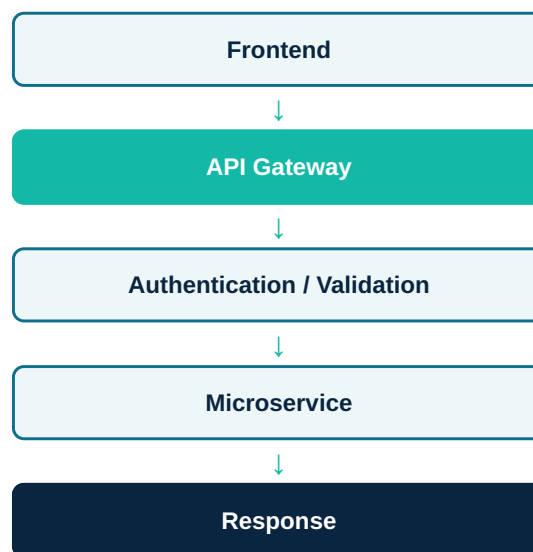
API Gateway

The API Gateway is an Express.js service listening on port **5000**. It is the single, central entry point through which the React frontend communicates with every backend microservice.

Documented Responsibilities

- Central entry point for all frontend API communication
- Rate limiting of incoming requests
- CORS configuration for cross-origin access from the frontend
- Helmet security headers applied to all responses
- Request routing to the appropriate downstream microservice, where documented

Request Flow



Gateway Route Reference

Endpoint details should be added from the complete API specification. The project overview establishes the Gateway's responsibilities (routing, rate limiting, CORS, Helmet headers) but does not list the individual proxied routes.

SECTION 07

Authentication Service

The Authentication Service (Node.js / Express, port **5001**) owns user identity for the platform.

Responsibilities

- Authentication of citizens, organizations, and admins
- Management of user identity
- Password security via bcrypt hashing
- Issuance and validation of JWTs
- Enforcement of role-based authorization

Endpoint Reference

Method	Endpoint	Description	Authentication	Role
Endpoint details should be added from the complete API specification.				

NOTE The source project overview confirms that the Authentication Service is responsible for identity, bcrypt password hashing, and JWT issuance, but it does not enumerate individual route paths (for example, login, registration, or token refresh), request bodies, or response payloads. No endpoints are invented here; the table above intentionally reflects that gap.

SECTION 08

Resource Service

The Resource Service (Node.js / Express, port **5002**) manages the core content of the HealthBridge platform.

Responsibilities

- Health resources
- Health articles
- Resource status tracking
- Organization-created resources
- Resource approval / rejection workflows

Endpoint Reference

Method	Endpoint	Description	Authentication / Role
Endpoint details should be added from the complete API specification.			

Resource Endpoint (template)

HTTP Method: Endpoint details should be added from the complete API specification.

URL: Endpoint details should be added from the complete API specification.

Description: Endpoint details should be added from the complete API specification.

Authentication: Endpoint details should be added from the complete API specification.

Authorization / Role: Endpoint details should be added from the complete API specification.

Request Parameters: Endpoint details should be added from the complete API specification.

Request Body: Endpoint details should be added from the complete API specification.

Response: Endpoint details should be added from the complete API specification.

Possible Errors: Endpoint details should be added from the complete API specification.

NOTE The project overview establishes what the Resource Service is responsible for — resources, articles, status, and approval workflows — but does not provide the underlying route paths or payload schemas needed to document individual endpoints.

SECTION 09

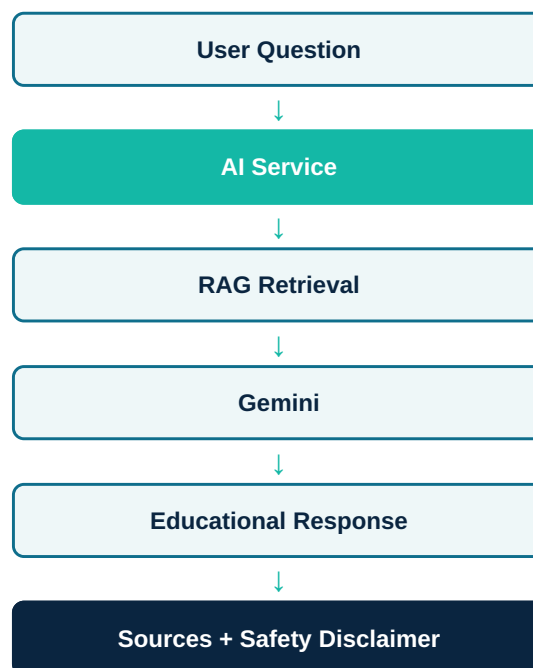
AI Service

The AI Service (Node.js / Express, port **5003**) powers HealthBridge's AI-driven features, including the Health Education Assistant, the RAG pipeline, and the Health Resource Agent.

Documented Capabilities

- Integration with the Google Gemini API
- Health education responses grounded via RAG
- AI prompt templates tailored to health education
- Health safety disclaimers attached to AI output
- A fallback mode that activates when a Gemini API key is unavailable

Response Flow



AI Endpoint Reference

Endpoint details should be added from the complete API specification. The project overview documents the AI Service's capabilities and its Gemini fallback behavior, but does not specify concrete route paths or payload schemas.

SECTION 10

Health Resource Agent

The Health Resource Agent is a tool-using component of the AI Service. Rather than answering purely from a language model, it can invoke a small set of defined tools to retrieve real platform data before responding.

Available Tools

search_resources

Search available health resources.

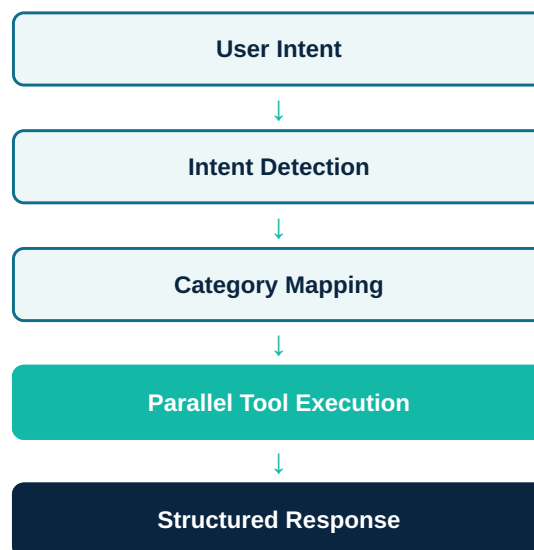
search_health_articles

Search health education articles.

get_resource_categories

Retrieve resource categories.

Execution Flow



The structured response returned by the agent may contain:

- Resources
- Articles
- A tool call log

Endpoint details should be added from the complete API specification. Each tool (`search_resources` , `search_health_articles` , `get_resource_categories`) should be mapped to its corresponding backend endpoint once that mapping is available in the full API specification.

SECTION 11

Python FastAPI Service

The Python service (port **8000**) is built with Python 3.11+, FastAPI, and Pydantic, using an object-oriented architecture.

Core Components

HealthResourceAnalyzer

Performs resource classification and relevance scoring.

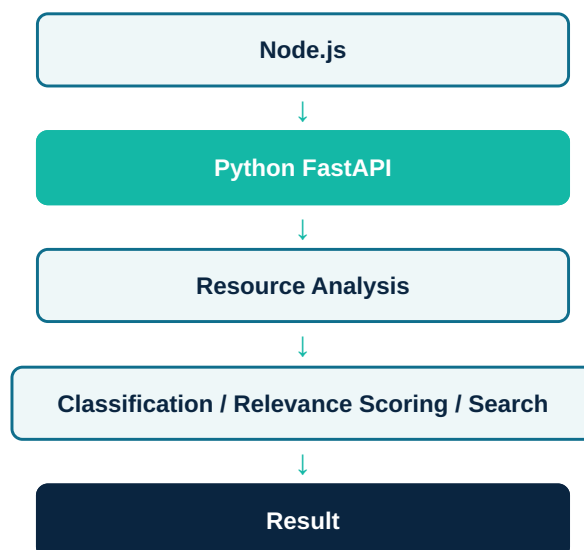
ContentClassifier

Handles content categorization and keyword extraction.

ResourceSearchEngine

Performs search and ranking over resources.

Integration Flow



FastAPI Route Reference

Endpoint details should be added from the complete API specification. The project overview names the three core Python components and their responsibilities but does not document the FastAPI route paths that expose them.

SECTION 12

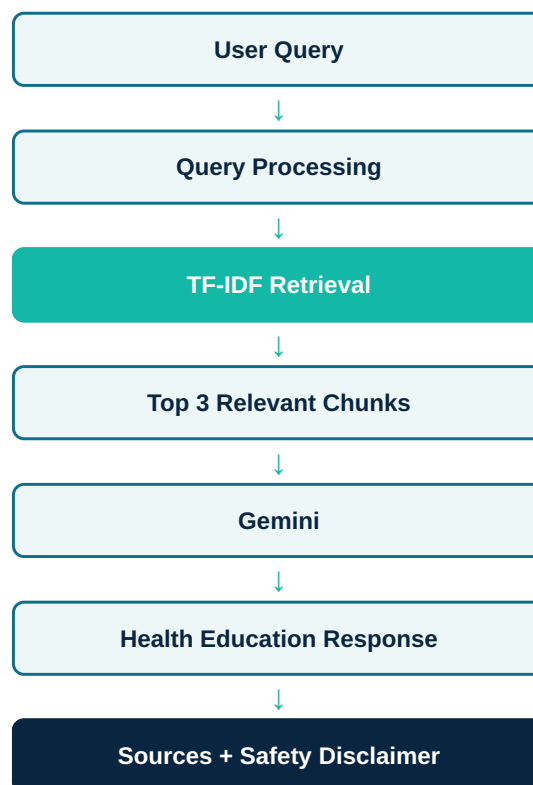
RAG System

HealthBridge grounds its AI-generated health education content using a Retrieval-Augmented Generation (RAG) pipeline.

Documented Implementation

- Seven Markdown knowledge-base files form the retrieval corpus
- TF-IDF is used for retrieval over that corpus
- The top three most relevant chunks are selected per query
- Google Gemini generates the final response from the retrieved context
- Sources are displayed in the UI alongside the generated response

RAG Pipeline



IMPORTANT RAG grounds AI responses in HealthBridge's own knowledge base; it does not guarantee medical accuracy. Every AI-generated response is expected to carry a safety disclaimer and cited sources.

SECTION 13

Request & Response Standards

All HealthBridge services communicate using conventional HTTP semantics and JSON payloads.

General Conventions

- Standard HTTP methods (GET, POST, PUT, DELETE) express intent
- Authentication is carried via HTTP-only cookies / JWTs rather than manual header management by clients
- Request bodies are JSON-encoded
- Responses are JSON-encoded
- `Content-Type: application/json` is used for JSON payloads
- Requests are validated before being processed (see Section 15, Security)

Documented Request / Response Shape

Where a documented endpoint exists, this manual represents it as:

```
METHOD /endpoint
```

Request

```
{  
  "field": "value"  
}
```

Response

```
{  
  "success": true,  
  "data": {}  
}
```

NOTE These are illustrative shapes only. Actual field names are never invented; they are populated from documented endpoints where available, and left as placeholders otherwise.

SECTION 14

Error Handling

IMPORTANT The provided project overview does not define a complete standardized error-response schema.

Rather than presenting invented behavior as implemented behavior, this section provides a placeholder structure that the complete API specification should confirm or replace.

Placeholder Error Structure

```
{
  "success": false,
  "error": {
    "message": "Human-readable description of what went wrong",
    "code": "Documented error code (to be confirmed)"
  }
}
```

Placeholder Status Code Reference

Status Code	Meaning
<i>Endpoint details should be added from the complete API specification.</i>	

When the complete API specification becomes available, this section should be updated with the actual error envelope, status codes, and per-service error conventions used across the Gateway, Auth Service, Resource Service, AI Service, and Python FastAPI service.

SECTION 15

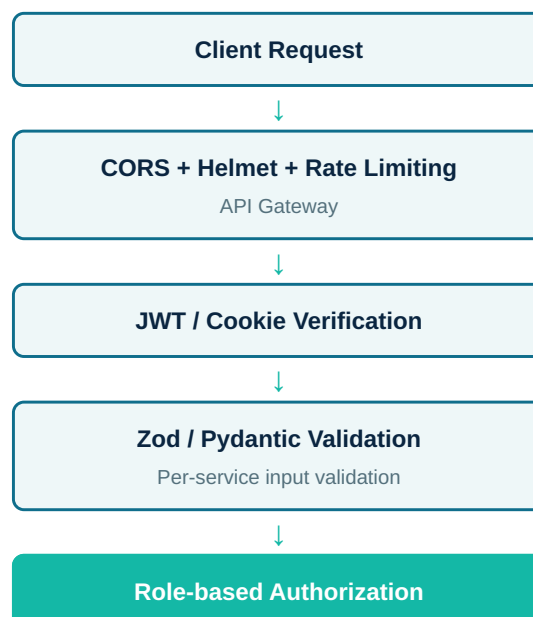
Security

HealthBridge applies defense-in-depth across the request lifecycle, from credential storage through to runtime request handling.

Implemented Mechanisms

Mechanism	Purpose
bcrypt (12 rounds)	Password hashing
JWT	Stateless session authentication
HTTP-only cookies	Prevents client-side script access to session tokens
Role-based authorization	Server-side enforcement of Citizen / Organization / Admin permissions
CORS	Restricts which origins may call the API
Helmet	Applies secure HTTP headers
API Gateway rate limiting	Mitigates abuse and brute-force attempts
Zod validation	Request validation in Node.js services
Pydantic validation	Request validation in the Python FastAPI service
No secrets committed to Git	Credentials and keys are kept out of version control
<code>.env</code> configuration	Environment-specific secrets and configuration
AI health disclaimers	Reduces risk of AI output being mistaken for medical advice

Security Architecture





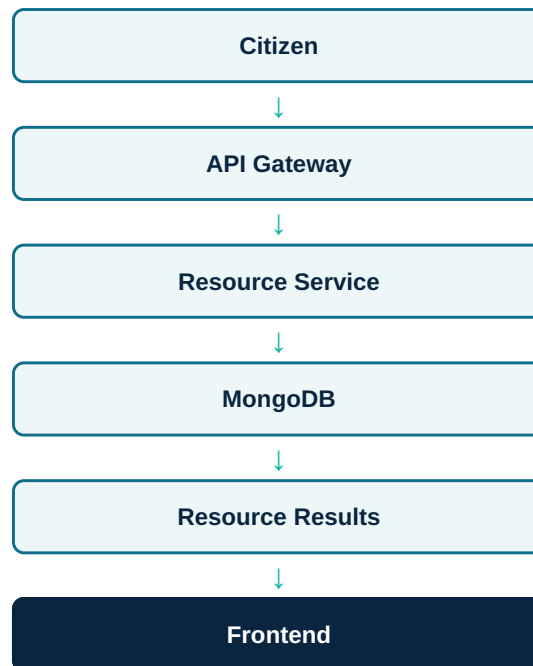
Request Processed

SECTION 16

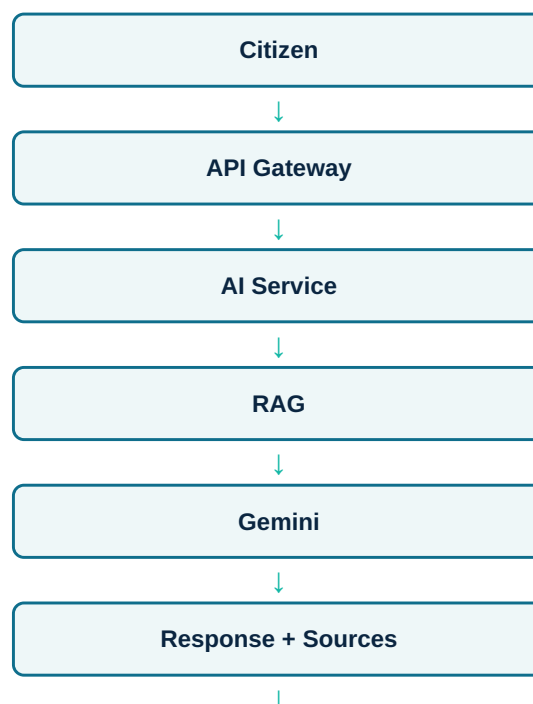
Example API Workflows

The following workflows describe how a request moves through the HealthBridge system for common, documented scenarios. Each step reflects only functionality described in the project overview.

Workflow 1 — Citizen Searches for a Health Resource

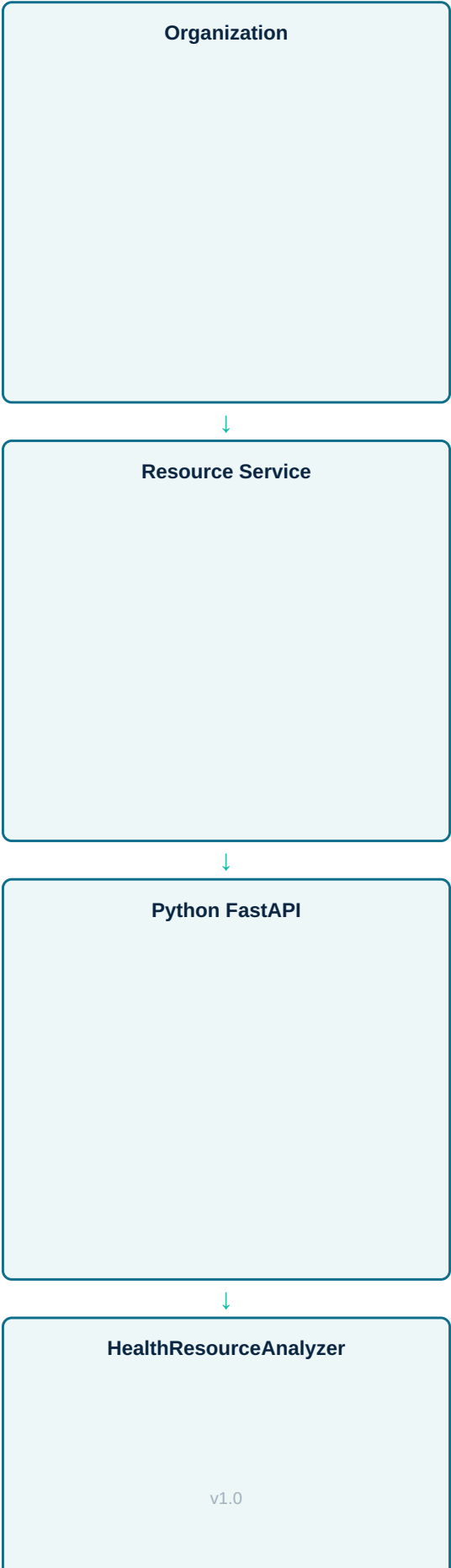


Workflow 2 — Citizen Asks an AI Health Question



Frontend

Workflow 3 — Health Resource Analysis





Workflow 4 — Resource Approval



SECTION 17

Environment Configuration

The following environment variables configure the HealthBridge services in the documented reference setup.

Variable	Description	Default
<code>MONGODB_URI</code>	MongoDB connection string	<code>mongodb://localhost:27017/healthbridge</code>
<code>JWT_SECRET</code>	JWT signing secret	Set in <code>.env</code>
<code>GATEWAY_PORT</code>	API Gateway port	5000
<code>AUTH_SERVICE_URL</code>	Auth service URL	<code>http://localhost:5001</code>
<code>RESOURCE_SERVICE_URL</code>	Resource service URL	<code>http://localhost:5002</code>
<code>AI_SERVICE_URL</code>	AI service URL	<code>http://localhost:5003</code>
<code>PYTHON_SERVICE_URL</code>	Python service URL	<code>http://localhost:8000</code>
<code>LLM_API_KEY</code>	Google Gemini API key	Empty = fallback mode
<code>VITE_API_URL</code>	Frontend API URL	<code>http://localhost:5000</code>
<code>FRONTEND_URL</code>	Frontend URL	<code>http://localhost:5173</code>

IMPORTANT No real secrets are included in this document. `JWT_SECRET` and `LLM_API_KEY` must be set per-environment in a local `.env` file that is never committed to version control.

SECTION 18

Running the API

Requirements

- Node.js 18+
- Bun
- Python 3.11+
- Docker & Docker Compose
- Google Gemini API key

Recommended Setup

The recommended way to run HealthBridge locally is via Docker Compose, which brings up the frontend, API Gateway, all three Node.js microservices, the Python FastAPI service, and MongoDB together.

```
docker compose up --build
```

Endpoint details should be added from the complete API specification. Specific database seeding commands and per-service startup scripts beyond the Docker Compose entry point were not included in the provided project overview.

SECTION 19

API Service Ports

Component	Port
Frontend	5173
API Gateway	5000
Auth Service	5001
Resource Service	5002
AI Service	5003
Python Service	8000
MongoDB	27017

Service Map

Frontend :5173	API Gateway :5000
Auth Service :5001	Resource Service :5002
AI Service :5003	Python Service :8000
MongoDB :27017	

SECTION 20

Future API Improvements

NOTE Everything in this section is a planned or potential improvement, not currently implemented functionality.

- Real-time notifications
- User reviews and ratings
- Multi-language support
- Mobile application
- Advanced NLP for intent detection
- Vector database for improved RAG
- Automated resource verification
- Detailed analytics dashboard

Each of these would extend the API surface described in this manual: real-time notifications and mobile support would introduce new transport layers alongside the existing REST API; a vector database would likely replace or augment the current TF-IDF retrieval step in the RAG pipeline; and advanced NLP would extend the intent detection stage of the Health Resource Agent described in Section 10.

SECTION 21

Conclusion

The HealthBridge API architecture provides the foundation for:

- Secure authentication
- Community health resource management
- Health education
- AI-powered assistance
- RAG-based information retrieval
- Intelligent resource discovery
- Microservice-based scalability

Together, the API Gateway, Auth Service, Resource Service, AI Service, and Python FastAPI service form a modular backend that keeps authentication, content management, and AI functionality cleanly separated while still working together through a single, consistent API surface.

**HealthBridge API — Connecting users, health resources, and AI
through a modular backend architecture.**
